

国家级精品课程
山东省一流本科课程



程序设计基础（C语言）

信息科学与工程学院

潘玉奇

第5章 函数

- ◆ 5.1 函数的引出
- ◆ 5.2 函数定义与调用
- ◆ 5.3 函数参数传递
- ◆ 5.4 函数的嵌套调用
- ◆ 5.5 递归与分治算法
- ◆ 5.6 局部变量和全局变量
- ◆ 5.7 变量的存储类别

5.1 函数的引出

在前面的程序设计中，曾多次使用系统提供的函数，用printf()函数实现数据的输出，用sqrt()函数进行开方运算等。

这些函数功能单一，使用方便，能有效减少程序设计的工作量。实际上，我们也可以自己编写实现具体功能的函数(自定义函数)，使程序结构清晰、实现共享。

【问题描述5.1】 设信息学院2年级有5个班，试找出5个班中最高的英语四级成绩平均分；找出该年级中学生英语成绩的最高分。

分析：

先计算每个班的平均成绩，放入包含5个元素的数组；再找出每个班的最高分，放入另一个包含5个元素的数组；然后分别进行比较，找出平均分最高的班级和分数最高的学生。

5.1 函数的引出

【问题描述5.1】算法描述：

step1: 定义变量Class[50], ClaAverage[5], ClaMax[5], AverMax, AllMax

step2: for(i=1; i<=5; i++) //分别对5个班级进行以下操作

{ ① 输入第i个班中每个同学的成绩, 将其存放到Class[50]

② 计算第i个班的平均分Average, 将其存放到ClaAverage[i]中

③ 找出第i个班中的最高分max, 将其存放到ClaMax[i]中

}

step3: 在数组ClaAverage[5]中找出最高分AverMax

step4: 在数组ClaMax[5]中找出最高分AllMax

step5: 输出AverMax、AllMax

注意：算法中多次用到查找最高分。

不管有多少数据，查找最高分可以采用相同的查找算法。

因此可以编写一个实现查找最高分功能的函数。

5.1 函数的引出

【例5.1】编程求解 $C_n^m = \frac{n!}{m!(n-m)!}$

分析:

求解公式最重要的就是计算阶乘，分别计算出n!, m!, (n-m)!, 再按公式进行除法运算即可得到结果。

c1=n! c2=m!
c3=(n-m)! c=c1/(c2*c3)

step1: 输入m和n

step2: 计算n!

step3: 计算m!

step4: 计算(n-m)!

step5: 计算n!/(m!*(n-m)!)

step6: 输出计算结果

```
#include<stdio.h>
int main ( )
{   int i, m, n;
    double c,c1=1,c2=1,c3=1;
    scanf("%d%d", &m, &n);
    for( i=1; i<=n; i++ )
        c1=c1*i;
    for( i=1; i<=m; i++ )
        c2=c2*i;
    for( i=1; i<=(n-m); i++ )
        c3=c3*i;
    c=c1/(c2*c3);
    printf("%lf\n", c);
    return 0;
}
```

5.1 函数的引出

【例5.1】编程求解 $C_n^m = \frac{n!}{m!(n-m)!}$

分析:

三次计算阶乘，除了阶乘的次数不同，代码是一样的。

因此可将计算阶乘的代码编写为函数。

```
double factorial (int x)
{   int i;
    double f=1;
    for( i=1; i<=x; i++ )
        f=f*i;
    return(f);    //返回计算结果
}
```

```
#include<stdio.h>
double factorial (int x)
{   int i;    double f=1;
    for( i=1; i<=x; i++ )
        f=f*i;
    return(f);
}

int main( )
{   int m, n; double c,c1,c2,c3;
    scanf("%d%d", &m, &n);
    c1= factorial (n);
    c2= factorial (m);
    c3= factorial (n-m);
    c=c1/(c2*c3);
    printf("%.2lf\n", c);
    return 0;
}
```



5.1 函数的引出

➤ 有关C 函数的说明

1. 在C语言中除main函数外，所有函数都是平行的。
每个函数相互独立，即函数不能嵌套定义，可以相互调用，但任何函数都不能调用main函数。
2. 一个主调函数可以调用多个被调函数，同一个函数也可以被一个或多个函数调用任意多次。
3. 从用户使用角度函数分两类：标准函数和用户自定义函数。
标准函数即库函数，它是由系统提供的，
用户自定义函数是用户根据需要自己定义的函数。



5.2 函数的定义与调用

5.2.1 函数的定义与调用

1. 函数定义的一般形式

数据类型 函数名(数据类型 形式参数1, ..., 数据类型 形式参数n)
{
 <说明语句>
 <执行语句>
}

说明:

- ① 数据类型，函数返回值的数据类型；
- ② 函数名，为函数起的名字，应符合标识符的命名规则；
- ③ 形式参数表，定义函数所接受的参数（变量）类型和个数；
- ④ { }部分是函数的函数体，其编写方法与主函数一样。

5.2.1 函数的定义与调用

2. 函数调用的一般形式

函数名(实际参数1, 实际参数2, ..., 实际参数n);

说明:

- ① 函数定义时，每个形式参数都要定义数据类型;
- ② 函数调用时，如有多个实际参数，则各参数间用逗号分开;
- ③ 实际参数是传递给被调用函数的值，可以是常量、变量、表达式或函数调用，但是其值必须确定。

实际参数和形式参数类型应一致，顺序应一一对应。

5.2.1 函数的定义与调用

【例5.2】编程求两个数中较大的数。

分析：将比较两个数，选出其中较大的数的过程用函数Max实现，而输入两个数和输出结果在main函数中实现。

注意：

Max函数中用于比较的两个数需要从main中获得，因此应定义两个参数，而Max函数中求得的较大数也必须返回给main输出。

```
#include<stdio.h>
int main()
{  int a, b, c;
    scanf("%d%d", &a, &b);
    if (a>b) c=a;
    else c=b;
    printf("%d\n", c);
    return 0;
}
```

定义一个函数

```
int Max(int a, int b)
{  int c;
    if (a>b) c=a;
    else c=b;
    return(c);
}
```

5.2.1 函数的定义与调用

【例5.2】参考程序1

```
#include<stdio.h>
```

```
//函数定义，a,b为形参，返回值为int型
```

```
int Max(int a, int b)
```

```
{ int c;
```

```
  if (a>b) c=a;
```

```
  else c=b;
```

```
  return(c); //返回变量c的值
```

```
}
```

```
int main( )
```

```
{ int x, y, z;
```

```
  scanf("%d%d", &x, &y);
```

```
  z=Max(x, y); //调用函数
```

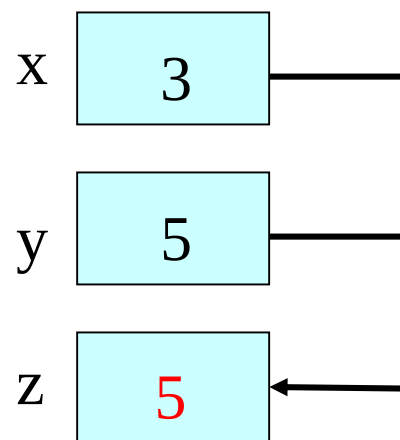
```
  printf("maxmum=%d\n", z);
```

```
  return 0;
```

```
}
```

假设输入为: 3 5

main函数



5.2.1 函数的定义与调用

```
#include<stdio.h>
```

```
int Max(int a, int b)  //函数定义
```

```
{  int c;  
    if (a>b) c=a;  
    else c=b;  
    return(c);    //返回变量c的值  
}
```

```
int main( )
```

```
{  int  x, y, z;  
    scanf("%d%d", &x, &y);  
    z=Max(x, y);    //调用函数，实参为变量  
    printf("maxmum=%d\n", z);  
    z=Max(3, 5);    //调用函数，实参为常量  
    printf("maxmum=%d\n", z);  
    z=Max(x+3, y*2 );    //调用函数，实参为表达式  
    printf("maxmum=%d\n", z);  
    z=Max(9, Max(x, y) );    //调用函数，第2个实参为函数调用  
    printf("maxmum=%d\n", z);  
    return 0;  
}
```

5.2.1 函数的定义与调用

3. 函数的返回值

(3) 定义函数时指定的函数返回值类型应该和return语句中的表达式的类型保持一致。如果两者不一致，则以函数返回值类型为准，对于数值型的数据，系统会自动进行类型转换。

【例5.2】参考程序2--函数返回值类型与return语句表达式类型不同

```
int Max(double a, double b) //定义函数值为int型
{
    double c; //定义变量c为double型
    c=a>b?a : b; //将例5.2的if语句改为条件表达式
    return(c); //返回c的值，注意c为double型，而函数值为int型
}

int main( )
{
    double x, y;    int z;
    scanf("%lf%lf", &x, &y);
    z=Max(x, y);
    printf("maxmum=%d\n", z);
    return 0;
}
```

输入： 1.6 3.8✓
输出： maxmum=3

5.2.1 函数的定义与调用

3. 函数的返回值

- (1) 函数的值是通过**return**语句返回到主调函数，**return**语句的功能是计算表达式的值，并返回给主调函数。

return语句的一般形式：**return (表达式);**
或 **return 表达式;**

- (2) 函数中可以有多条**return**语句，但每次调用只可能有一个**return**语句被执行。

例5.2中的Max函数还可以定义成如下形式

```
int Max(int a, int b)
{
    if (a>b) return a;
    else return b;
}
```



5.2.1 函数的定义与调用

3. 函数的返回值

(4) 如果函数没有或者不需要返回计算结果，可以明确将函数的返回值类型定义为空类型（即**void**）。

【例5.3】编写函数计算n个数的平均数，在main中输入n的值。

分析：

如果想在main函数中输出计算结果，应该将函数定义成有返回值的；如果直接在函数内输出计算结果，则可以将函数定义成无返回值的，因题目要求在main中输入n，所以函数应该定义一个参数。

5.2.1 函数的定义与调用

【例5.3】参考程序1--函数无返回值

```
#include<stdio.h>
```

```
void Average(int n) //定义计算平均数函数，参数n表示整数个数
```

```
{ int i, x;      double fAve, fSum=0.0;
```

```
  for(i=1; i<=n; i++)
```

```
  {  scanf("%d", &x);
```

```
    fSum=fSum+x;
```

```
  }
```

```
  fAve=fSum/n;
```

```
  printf("ave=%.2lf\n", fAve); //输出计算结果
```

```
}
```

```
int main()
```

```
{ int n;
```

```
  printf("计算n个整数的平均数，请输入n:");
```

```
  scanf("%d", &n);
```

```
  Average(n); //调用Average函数
```

```
  return 0;
```

```
}
```

5.2.1 函数的定义与调用

【例5.3】参考程序2--函数有返回值

```
#include<stdio.h>
```

```
double Average(int n) //定义计算平均数函数，参数n表示整数个数
```

```
{ int i, x;      double fAve, fSum=0.0;
```

```
  for(i=1; i<=n; i++)
```

```
  {  scanf("%d", &x);
```

```
    fSum=fSum+x;
```

```
  }
```

```
  fAve=fSum/n;
```

```
  return(fAve);
```

```
}
```

```
int main()
```

```
{ int n;      double ave;
```

```
  printf("计算n个整数的平均数，请输入n:");
```

```
  scanf("%d", &n);
```

```
  ave=Average(n); //调用Average函数
```

```
  printf("ave=%.2f\n", ave); //输出
```

```
  return 0;
```

```
}
```

一般函数实现计算功能时，推荐写成有返回值的

5.2.1 函数的定义与调用

3. 函数的返回值

- (5) 如果函数不需要从主调函数传入数据，在定义函数时可以有形式参数，这样的函数称为“无参函数”。定义无参函数时，函数名后的小括号中可以写void。

定义无参函数的形式：
数据类型 函数名(void)
{
 <说明语句>
 <执行语句>
}

调用无参函数的形式为：
函数名();

注意：
不能省略函数名后的小括号

5.2.1 函数的定义与调用

【例5.4】从键盘输入一串字符，输入‘#’时结束，将字符串的大写字母转换为小写字母输出，其它字符直接输出。

分析：

编写一个函数实现将大写字母转换为小写字母，如果输入一串字符的操作在main中实现，则该函数需要定义参数；如果直接在函数中实现输入一串字符的操作，就不需要定义参数了

5.2.1 函数的定义与调用

【例5.4】 参考程序

```
#include<stdio.h>
```

```
void Change(void)    //定义Change函数，该函数无参数也无返回值
```

```
{  char ch;
   ch=getchar();
   while(ch!='#' )
   {   if(ch>='A' && ch<='Z')
       ch=ch+32;
       putchar(ch);
       ch=getchar();
   }
}
```

```
int main( )
{
    Change( );          //调用Change函数
    return 0;
}
```

5.2.1 函数的定义与调用



【练习5.1】

编写函数，计算一个整数各位数字之和，
在main函数中输入这个整数，并输出计算结果。

例如，整数123，各位数字之和为 $1+2+3=6$ 。

提示：定义一个参数传递整数，函数有一个整型返回值

5.2.2 函数声明与函数原型

C语言中在函数调用之前应当对所调用的函数进行声明，指出该函数的返回值的类型以及形参的个数和类型，编译器据此信息对函数调用进行语法检查，保证形参和实参的个数和类型的一致性，保证返回值的使用正确性。

函数声明不是函数定义，函数定义除了描述函数的基本特征外，核心内容是对函数功能的具体描述；而函数声明只是描述函数的基本特征(包括函数名，函数类型，形参表)，即函数原型。

函数原型的一般形式：

函数类型 函数名(形参类型1 形参名1, ..., 形参类型n 形参名n);

5.2.2 函数声明与函数原型

说明:

- ① 在函数声明中，由于编译系统不检查参数名，因此形参表中可以只写形参的数据类型，而不写形参名，从而可以简化为以下形式：

函数类型 函数名(形参类型1, ..., 形参类型n);

- ② 函数声明中，函数类型、函数名、参数个数、参数类型和参数顺序应与函数定义保持一致。

```
#include <stdio.h>
float count (int, float);
int main()
{ ...
}
float count(int x,float y)
{ ...
}
```

5.2.2 函数声明与函数原型

③ 函数声明的位置:

一种是在主调函数中对被调函数进行函数声明;

另一种是在所有函数的外部进行函数声明(推荐使用)

(1) 在主调函数中声明

(2) 在所有函数的外部进行声明

```
#include <stdio.h>
```

```
int main ( )
```

```
{ int f (int x, int y);
```

函数声明

```
int a, b, c;
```

```
scanf("%d%d", &a, &b);
```

```
c = f(a, b);
```

```
printf("%d\n", c);
```

```
return 0;
```

```
}
```

```
int f (int x, int y)
```

```
{ x = x+2; y = y*2;
```

```
return( x+y );
```

```
}
```

函数定义

```
#include <stdio.h>
```

```
int f (int x, int y);
```

```
int main ( )
```

```
{ int a, b, c;
```

```
scanf("%d%d", &a, &b);
```

```
c = f(a, b);
```

```
printf("%d\n", c);
```

```
return 0;
```

```
}
```

```
int f (int x, int y)
```

```
{ x = x+2; y = y*2;
```

```
return( x+y );
```

```
}
```

函数定义



5.2.2 函数声明与函数原型

【例5.5】编程求排列公式 $P_n^m = \frac{n!}{(n-m)!}$ 组合公式 $C_n^m = \frac{P_n^m}{m!}$

【例5.5】参考程序1--在主调函数中对被调函数进行函数声明

```
1. #include<stdio.h>
2. int main()
3. {
4.     double fP, fC;  int n, m;
5.     double Pfun(int n, int m);    //对Pfun函数进行函数声明
6.     double Factorial(int x);      //对Factorial函数进行函数声明
7.     scanf(" %d%d", &n, &m);
8.     fP=Pfun(n, m);
9.     fC=fP/Factorial(m);
10.    printf("P=%.2lf, C=%.2lf \n", fP, fC);
11. }
```



5.2.2 函数声明与函数原型

【例5.5】参考程序1--在主调函数中对被调函数进行函数声明

```
12. double Pfun(int n, int m)    //定义函数，计算排列公式P
13. {
14.     double r;
15.     double Factorial (int x); //对Factorial函数进行函数声明
16.     r=Factorial(n)/Factorial(n-m);
17.     return r;
18. }
19. double Factorial(int x)      //定义函数，计算x的阶乘
20. {
21.     double fValue=1;
22.     for(int i=1; i<=x; i++)
23.         fValue=fValue*i;
24.     return fValue;
25. }
```

注意：由于在main函数和Pfun函数中都要调用Factorial函数，所以在main和Pfun函数中都对Factorial函数进行了函数声明。



5.2.2 函数声明与函数原型

【例5.5】 参考程序2--在所有函数外部进行函数声明

```
#include<stdio.h>
double Pfun(int n, int m);    //对Pfun函数进行函数声明
double Factorial (int x);    //对Factorial函数进行函数声明
int main()
{  double fP, fC;  int n, m;
   scanf("%d%d",&n,&m);
   fP=Pfun(n, m);
   fC=fP/Factorial(m);
   printf("P=%.2lf, C=%.2lf \n", fP, fC);
   return 0;
}
double Pfun(int n, int m)
{  double r;
   r=Factorial(n)/Factorial(n-m);
   return r;
}
double Factorial(int x)
{  ..... }
```

说明：
程序在#include命令后对Pfun函数和Factorial 函数进行了函数声明，因此在main和Pfun函数中就不需要再对Factorial函数进行声明了。



5.2.2 函数声明与函数原型

➤ 以下几种情况可以省略主调函数中对被调函数的函数声明：

- ① 对标准函数的调用不需要进行函数声明，但必须在程序开头用**#include**命令把标准函数所在的头文件包含到本文件中。
- ② 如果被调函数的定义出现在主调函数之前，则在主调函数中可以不对被调函数进行声明而直接调用。
- ③ 如果在所有函数定义之前，在函数的外部已作了函数声明，则在各个主调函数中不必对所调用的函数再作声明。

5.3 函数参数传递

5.3.1 简单变量作函数参数

简单变量作函数参数时，参数传递数据是单向的，只能把主调函数的实参传给被调函数的形参（单向的数据传递）

说明：

- ① 实参可以是常量、变量、表达式或函数调用。在进行函数调用时，实参必须有确定的值，以便把这些值传递给形参。
- ② 只有在函数被调用时系统才会为形参分配内存单元，在函数调用结束时，系统会立即释放形参所占用的内存单元。因此，形参只在函数内部有效，在函数调用过程中，形参的值发生改变，不会影响实参，函数调用结束返回主调函数后则不能再使用该形参变量。

5.3.1 简单变量作函数参数

【例5.6】简单变量作函数参数，形参的变化不影响实参

```
#include <stdio.h>
```

```
int fun(int x, int y)
```

```
{ x = x+2; y = y*2;
```

```
printf("x=%d,y=%d\n", x, y);
```

```
return( x+y );
```

```
}
```

```
int main ( )
```

```
{ int a, b, c;
```

```
scanf("%d%d", &a, &b);
```

```
c = fun(a, b);
```

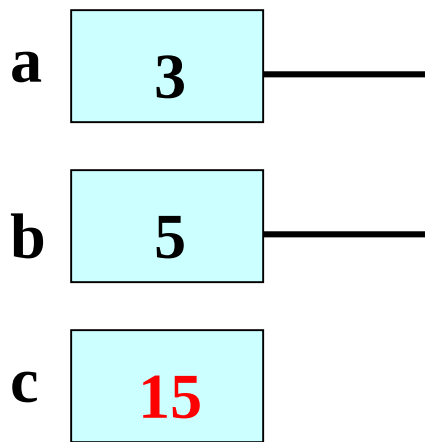
```
printf("a=%d,b=%d,c=%d\n", a, b, c);
```

```
return 0;
```

```
}
```

假设输入为: 3 5

main函数



输出:

x=5, y=10

a=3, b=5, c=15

函数调用过程:

- (1) 形参与实参各自占有一个独立的存储空间
- (2) 形参的存储空间在函数被调用时才分配
- (3) 函数返回时, 形参占据的临时存储区被撤消

5.3.2 数组作为函数参数

1. 数组元素作函数参数

数组元素就是下标变量，它与简单变量并无区别，因此它作函数实参的使用与简单变量是完全相同的，函数调用时，把数组元素的值传给形参，实现单向的值传递。

【例5.7】输入一个班的某门课程的成绩，将百分制成绩转换为五个等级，输出每个学生的百分制成绩和对应的等级
转换规则：

A—90~100； B—80~89； C—70~79； D—60~69； E— 0~59

分析：

- 将“成绩转换”单独编写一个函数，转换函数需要定义一个形参，其作用是从主调函数获得需要进行转换的百分制成绩，并将转换后的等级返回给主调函数，所以该函数应定义成有返回值的函数。
- 在main函数中实现成绩的输入和输出，根据题目要求应该定义两个数组，一个数组存放学生的百分制成绩(整型数组)，另一个数组存放成绩对应的等级(字符数组)。

5.3.2 数组作为函数参数

【例5.7】参考程序

```
#include<stdio.h>
#define N 30          //定义学生人数
char Change( int x); //函数声明

int main()
{   int iScore[N] , i;
    char chGrade[N];
    printf("输入%d个学生的成绩:", N);
    for(i=0; i<N; i++)
        scanf(" %d", &iScore[i]);
    for(i=0; i<N; i++)
        chGrade[i]=Change( iScore[i] );
    for(i=0; i<N; i++)
    {   printf("学生%d的成绩: %d, ", i+1, iScore[i]);
        printf("对应等级: %c\n", chGrade[i]);
    }
    return 0;
}
```

```
char Change( int x )
{   char g;
    switch( x/10 )
    {   case 10:
        case 9: g='A'; break;
        case 8: g='B'; break;
        case 7: g='C'; break;
        case 6: g='D'; break;
        default: g='E';
    }
    return(g);
}
```

5.3.2 数组作为函数参数

2. 数组名作函数参数

实际应用中，有时希望在执行被调函数后，能修改主调函数中一个或多个变量的值。如对一组数排序，如果将排序功能写成一个函数，希望在排序函数执行后，主调函数能得到排序后的结果，但是用简单变量或数组元素作函数参数都无法完成这个任务，这时就需要用数组名作函数参数。

因数组名是数组的首地址，所以数组名作函数参数时，不是把实参数组每个元素的值都赋给形参数组的各个元素，而是把实参数组的首地址传给形参数组。实际上，形参数组和实参数组拥有同一段内存空间，因此形参数组发生变化时实参数组也随之变化。

5.3.2 数组作为函数参数

【例5.8】用数组名作函数参数实现冒泡排序

```
#include <stdio.h>
```

```
#define N 6
```

```
int main( )
```

```
{ int a[N] , i , j , temp;
```

```
  for( i=0; i<N; i++)
```

```
    scanf("%d", &a[i] );
```

```
  for( i=0; i<N-1; i++)
```

```
    for( j=0; j<N-1-i; j++)
```

```
      if ( a[j]>a[j+1] )
```

```
        { temp=a[j] ;
```

```
          a[j]=a[j+1] ;
```

```
          a[j+1]=temp ;
```

```
        }
```

```
  for(i=0; i<N; i++)
```

```
    printf( "%3d", a[i]);
```

```
  return 0;
```

```
}
```

```
#include <stdio.h>
```

```
#define N 6
```

```
void sort(int a[N]);  函数声明
```

```
int main( )
```

```
{ int b[N], i ;
```

```
  for( i=0; i<N; i++)
```

```
    scanf("%d", &b[i] );
```

```
  sort( b );  函数调用
```

```
  for ( i=0; i<N; i++)
```

```
    printf( "%3d", b[i] );
```

```
  return 0;
```

```
}
```

```
void sort (int a[N])
```

```
{ int i, j, temp;
```

```
  for( i=0; i<N-1; i++)
```

```
    for( j=0; j<N-1-i; j++)
```

```
      if ( a[j]>a[j+1] )
```

```
        { temp=a[j];
```

```
          a[j]=a[j+1];
```

```
          a[j+1]=temp;
```

```
        }
```

```
}
```

函数定义

5.3.2 数组作为函数参数

【例5.9】求两个班级成绩的平均分，设两个班级的学生人数分别为30人和45人。

分析：

由于两个班级的人数不同，在定义求平均分函数时除了定义一个形参数组外，还应定义一个形参来传递人数，这样求平均分函数就可以求任何一个班级的平均分。

```
double Average(int iScore[ ], int n)
{
    double fAve, fSum=0;
    for (int i=0; i<n; i++)
        fSum=fSum+iScore[i];
    fAve=fSum/n;
    return fAve;
}
```



5.3.2 数组作为函数参数

【例5.9】 参考程序

```
1. #include <stdio.h>
2. double Average(int iScore[ ], int n);    //Average函数声明
3. int main()
4. {
5.     int i, iClass1[30], iClass2[45];
6.     double fAve1, fAve2 ;
7.     printf("输入班级1的成绩: \n");
8.     for (i=0; i<30 ; i++)
9.         scanf ("%d", &iClass1[i]);
10.    fAve1= Average (iClass1, 30);
11.    printf("输入班级2的成绩: \n");
12.    for (i=0; i<45 ; i++)
13.        scanf ("%d", &iClass2[i]);
14.    fAve2= Average (iClass2, 45);
15.    printf("班级1的平均分: ave1=%.2lf \n", fAve1);
16.    printf("班级2的平均分: ave2=%.2lf \n", fAve2);
17.    return 0;
18. }
```

第1次调用Average函数，
将数组iClass1 的首地址传给形参数组iScore

第2次调用Average函数，
将数组iClass2 的首地址传给形参数组iScore

5.3.2 数组作为函数参数

3. 多维数组名作为函数参数

多维数组名作函数参数，依然是将实参数组的首地址传给形参数组。

【例5.10】编写函数实现矩阵的转置，在main函数中进行矩阵的输入和转置后矩阵的输出。

$$a_{2 \times 3} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \xrightarrow{\text{a 转置后得到矩阵 b}} b_{3 \times 2} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

分析：

矩阵一般用二维数组存放，应该定义两个数组，一个是 $n \times m$ 的二维数组存放转置前的矩阵a，另一个则是 $m \times n$ 的二维数组存放转置后的矩阵b，而转置函数也需要定义两个形参数组。

5.3.2 数组作为函数参数

【例5.10】 参考程序

```
#include<stdio.h>
```

```
#define N 2
```

//定义数组一维长度

```
#define M 3
```

//定义数组二维长度

```
void Transpose ( int ta[N][M], int tb[M][N] )    //函数定义
```

```
{
```

```
    int i, j;
```

```
    for(i=0; i<N; i++)
```

```
        for(j=0; j<M; j++)
```

```
            tb[j][i]=ta[i][j];
```

```
}
```

在函数定义时，可以省去第一维的长度。
例如以下写法是合法的：

```
void Transpose(int ta[ ][M], int tb[ ][N])
```

//实现矩阵转置

5.3.2 数组作为函数参数

【例5.10】参考程序

```
int main( )
{
    int i, j, a[N][M]={0}, b[M][N]={0};    // a、b数组初始化
    printf("输入矩阵a(%d行,%d列): \n", N, M);
    for (i=0; i<N; i++)                    //输入数组a
        for (j=0; j<M; j++)
            scanf("%d", &a[i][j]);
    Transpose(a, b);                        //调用Transpose函数
    for (i=0; i<N; i++)                    //输出数组a
    {
        for (j=0; j<M; j++)
            printf("%3d", a[i][j]);
        printf("\n");
    }
    for (i=0; i<M; i++)                    //输出数组b
    {
        for (j=0; j<N; j++)
            printf("%3d", b[i][j]);
        printf("\n");
    }
    return 0;
}
```

将实参数组a和b的首地址传给形参数组ta和tb，在函数内对tb[j][i]的赋值，实际上就是对数组b[j][i]的赋值

5.3.2 数组作为函数参数



【练习5.4】编写两个函数分别实现矩阵的输入和输出，假设矩阵是 $n \times m$ 维的， n 和 m 都小于10，在`main`中调用输入、输出函数。



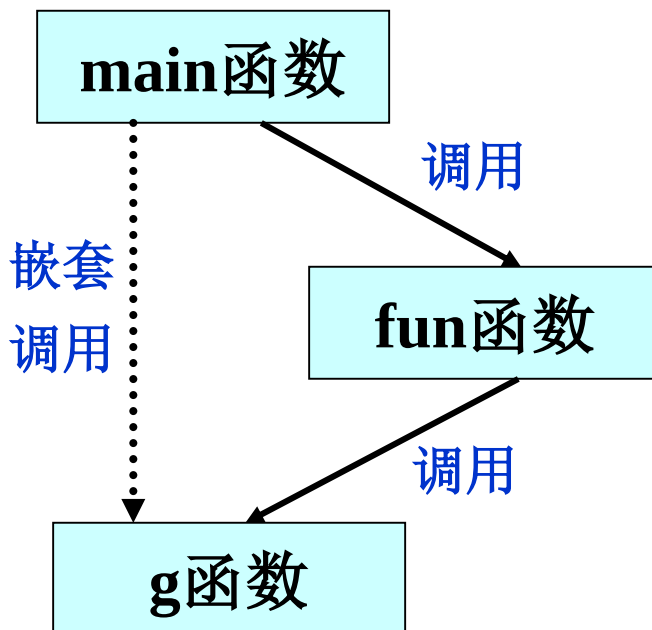
【练习5.5】在练习5.4的基础上增加一个函数，实现矩阵加法，在`main`中输入两个矩阵，输出这两个矩阵相加后的结果矩阵。

5.4 函数的嵌套调用

- 函数的嵌套调用是指主调函数调用被调函数，而在被调函数的执行过程中又调用另一个函数。

```
int main( )  
{  
    :  
    fun( ) ;  
    :  
}
```

```
void fun( void )  
{  
    :  
    g( ) ;  
    :  
}
```



5.4 函数的嵌套调用

```
#include <stdio.h>
```

```
void fa(void)
```

```
{
```

```
    putchar('a');
```

```
}
```

```
void fb(void)
```

```
{    fa();
```

```
    putchar('t');
```

```
}
```

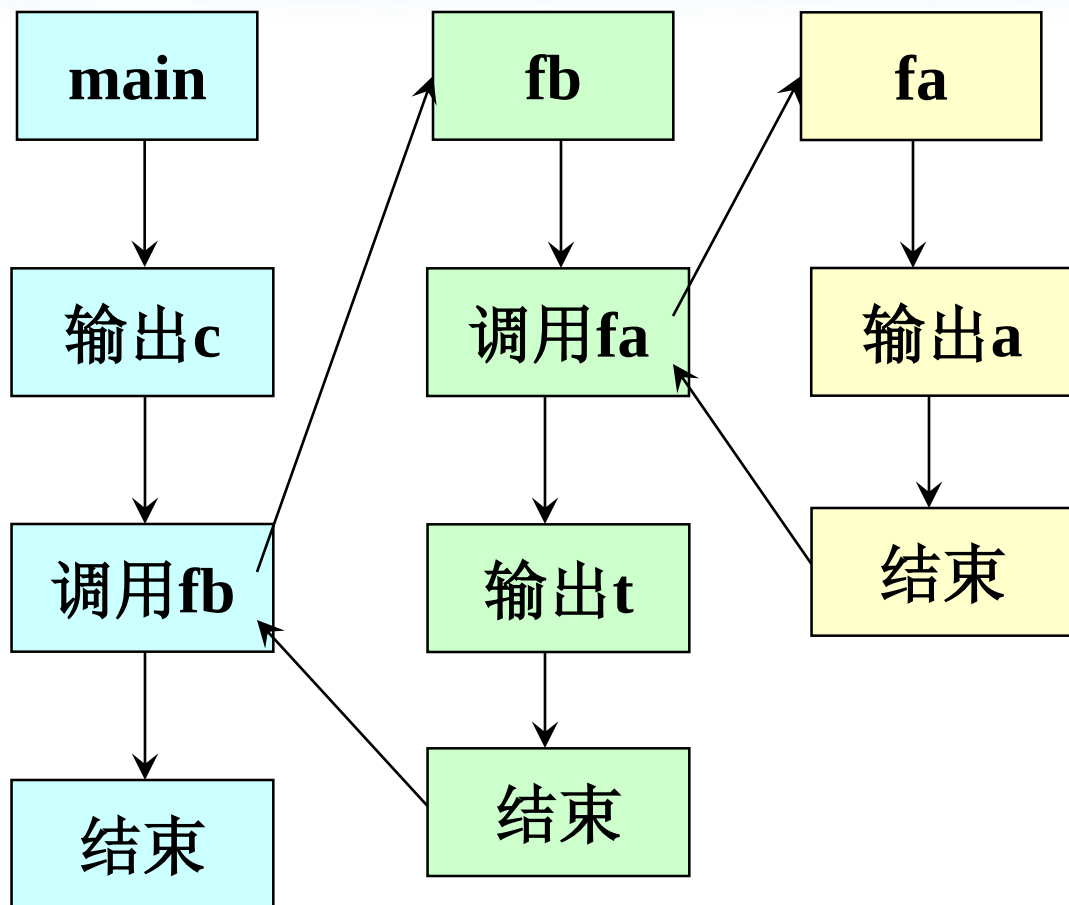
```
int main ()
```

```
{    putchar('c' );
```

```
    fb( );
```

```
    return 0;
```

```
}
```



输出: cat

5.4 函数的嵌套调用

【例5.12】编程求 $\sum_{x=1}^n x^k$ ，输入k和n的值

分析：

假设输入k=3, n=5, 即求 $1^3+2^3+3^3+4^3+5^3$

可分为以下4步求解：

- ① 输入n和k的值
- ② 乘方运算，即计算 x^k
- ③ 求和运算，即计算 $1^k+2^k+\dots+n^k$
- ④ 输出结果

需要编写三个函数：

乘方函数、求和函数、main 函数

注意：

- main函数调用求和函数，求和函数调用乘方函数
- main函数中将n和k作为实在参数，把它们的值传给求和函数



5.4 函数的嵌套调用

【例5.12】编程求 $\sum_{x=1}^n x^k$

```
int main( )           //主函数
{ int k, n;
  double sum;         // 结果可能很大, 因此用double型定义变量
  printf("input: k, n "); //提示信息
  scanf("%d%d", &k, &n);
  sum=sop(n, k);      //调用求和函数sop
  printf(" %.0lf\n", sum);
  return 0;
}
```



5.4 函数的嵌套调用

【例5.12】编程求 $\sum_{x=1}^n x^k$

```
double sop( int m, int t) //求和函数
{ int i;
  double sum, p;
  sum=0;           //初始化累加器
  for (i=1; i<=m; i++ )
  {   p=power(i, t);    //调用power函数，返回值赋给p
      sum=sum+p;        //累加
  }
  return (sum);    //返回sum的值
}
```



5.4 函数的嵌套调用

【例5.12】编程求 $\sum_{x=1}^n x^k$

```
double power( int p, int q )    //乘方函数
{
    int i;
    double product;             //乘方结果可能很大，因此用实型变量
    product=1;                   //累乘器设初值为1
    for(i=1; i<=q; i++)
        product=product*p;      //累乘
    return(product);             //返回product的值
}
```



【例5.12】完整程序

```
#include<stdio.h>
double sop( int m, int t);
double power(int p, int q);
int main( )
{ int k, n;  double sum;
  printf("input: k, n ");
  scanf("%d%d", &k, &n);
  sum=sop(n, k);
  printf("%.0lf\n", sum);
  return 0;
}
double sop( int m, int t)
{ int i; double sum, p;
  sum=0;
  for (i=1; i<=m; i++)
  { p=power(i, t);
    sum=sum+p;  }
  return (sum) ;
}
```

参数值的传递过程:

main		sop		power
n	→	m		
k	→	t	→	q
		i	→	p

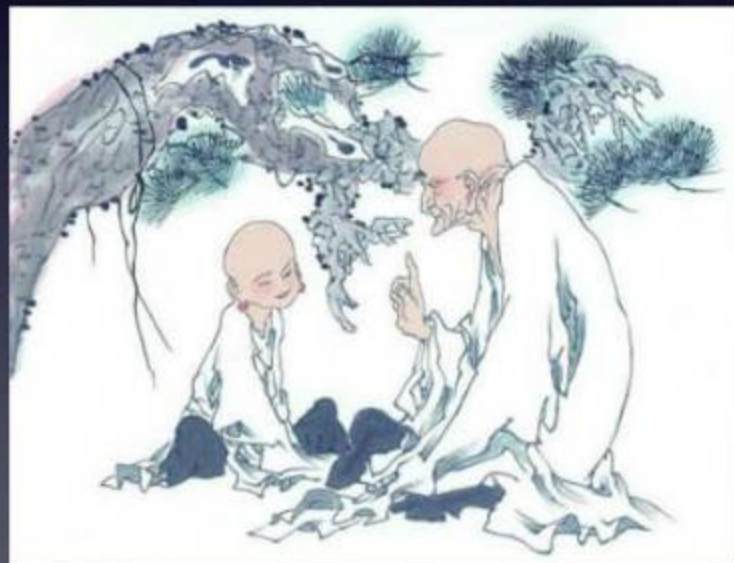
```
double power(int p, int q)
{ int i;
  double product;
  product=1;
  for(i=1; i<=q; i++)
    product=product*p;
  return(product);
}
```



5.5 递归与分治算法

- 从前有座山，山上有座庙，庙里有个老和尚和小和尚，有一天老和尚给小和尚讲故事，他说：
 - ◆ 从前有座山，山上有座庙，庙里有个老和尚和小和尚，有一天老和尚给小和尚讲故事，他说：
 - ▶ 从前有座山，山上有座庙，庙里有个老和尚和小和尚，有一天老和尚给小和尚讲故事，他说：

●



5.5 递归与分治算法

德罗斯特效应(Droste effect)是递归的一种视觉形式，是指一张图片的某个部分与整张图片相同，如此产生无限循环。



5.5.1 递归函数

- 一个函数在它的函数体内直接或间接调用它自身称为函数的递归调用，这种函数称为递归函数。
- 递归函数求解问题的基本原理是将复杂问题逐步简化，最终转化为一个可以直接求解的简单问题，这个简单问题的解决就意味着原复杂问题的解决。
- 递归求解问题时必须包括两部分：
 1. 递归的结束条件(即在此条件下可以直接求解问题)
 2. 求解问题的递归方式(即有明确的递归定义规则，可将原问题简化成较小规模的同类问题)



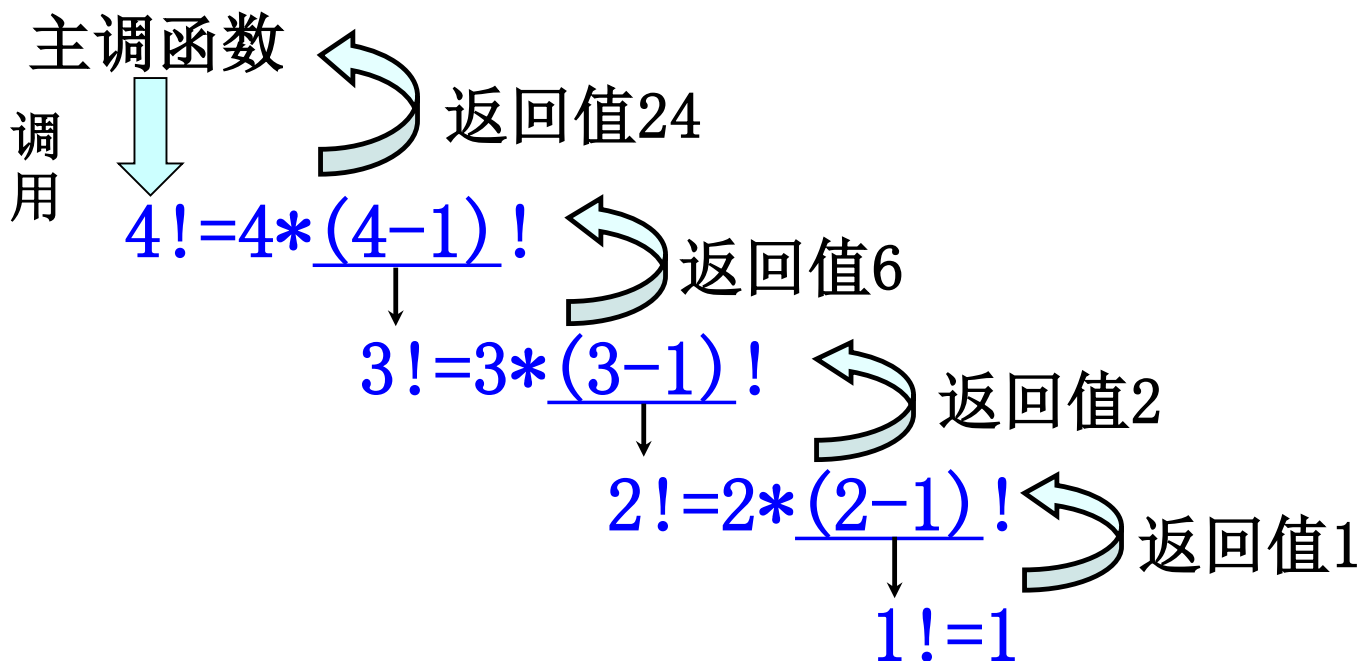
5.5.1 递归函数

【例5.13】用递归函数求n!

$$n! = \begin{cases} 1 & n=0,1 \\ n*(n-1)! & n>0 \end{cases}$$

递归的结束条件

递归方式



5.5.1 递归函数

【例5.13】用递归函数求n!

```
double Factorial ( int n )      // Factorial函数定义
{ double fValue;
  if ( n==0 || n==1 )          //递归的结束条件
    fValue =1;
  else
    fValue = n*Factorial ( n-1 ); //递归调用
  return(fValue);
}
```

- **存在问题:** 没有对n的合法性进行检查, 当 $n < 0$ 时会出错



【例5.13】 参考程序

```
#include <stdio.h>
#include <stdlib.h>           // exit函数的头文件
double Factorial ( int n )   // Factorial函数定义
{ double fValue;
  if ( n<0 )                 //对n进行合法性检查
  { printf("n<0, dataerror! "); exit(-1); } //用exit函数退出程序
  else
  { if ( n==0 || n==1 )      //递归的结束条件
    fValue =1;
    else fValue = n*Factorial ( n-1 ); //递归调用
  }
  return(fValue);
}

int main( )
{ int iNumber;      double y;
  scanf(" %d", &iNumber );
  y=Factorial (iNumber);
  printf(" %d!=%.0lf \n", iNumber , y );
  return 0;
}
```



5.5.1 递归函数

【例5.14】用递归函数求Fibonacci数列的第n项

Fibonacci数列: 1, 1, 2, 3, 5, 8, 13...

```
#include <stdio.h>
```

```
int main( )
```

```
{ int i, f1=1, f2=1, f3;
```

```
    printf(" %8d%8d", f1, f2);
```

```
    for ( i=3; i<=20; i++ )
```

```
    {
```

```
        f3=f1+f2;
```

```
        f1=f2;
```

```
        f2=f3;
```

```
        printf(" %8d", f3);
```

```
        if ( i%4==0) putchar('\n');
```

```
    }
```

```
    return 0;
```

```
}
```

迭代法在已知数列前2项的基础上, 从第3项开始, 依次向后计算, 得出数列的每一项

思考: 怎样用递归的方法求解?

5.5.1 递归函数

【例5.14】用递归函数求Fibonacci数列的第n项

分析：定义Fibonacci数列的递归数学模型

$$F(n) = \begin{cases} 1 & n=0,1 \\ F(n-1)+F(n-2) & n>1 \end{cases}$$

递归的结束条件

递归方式

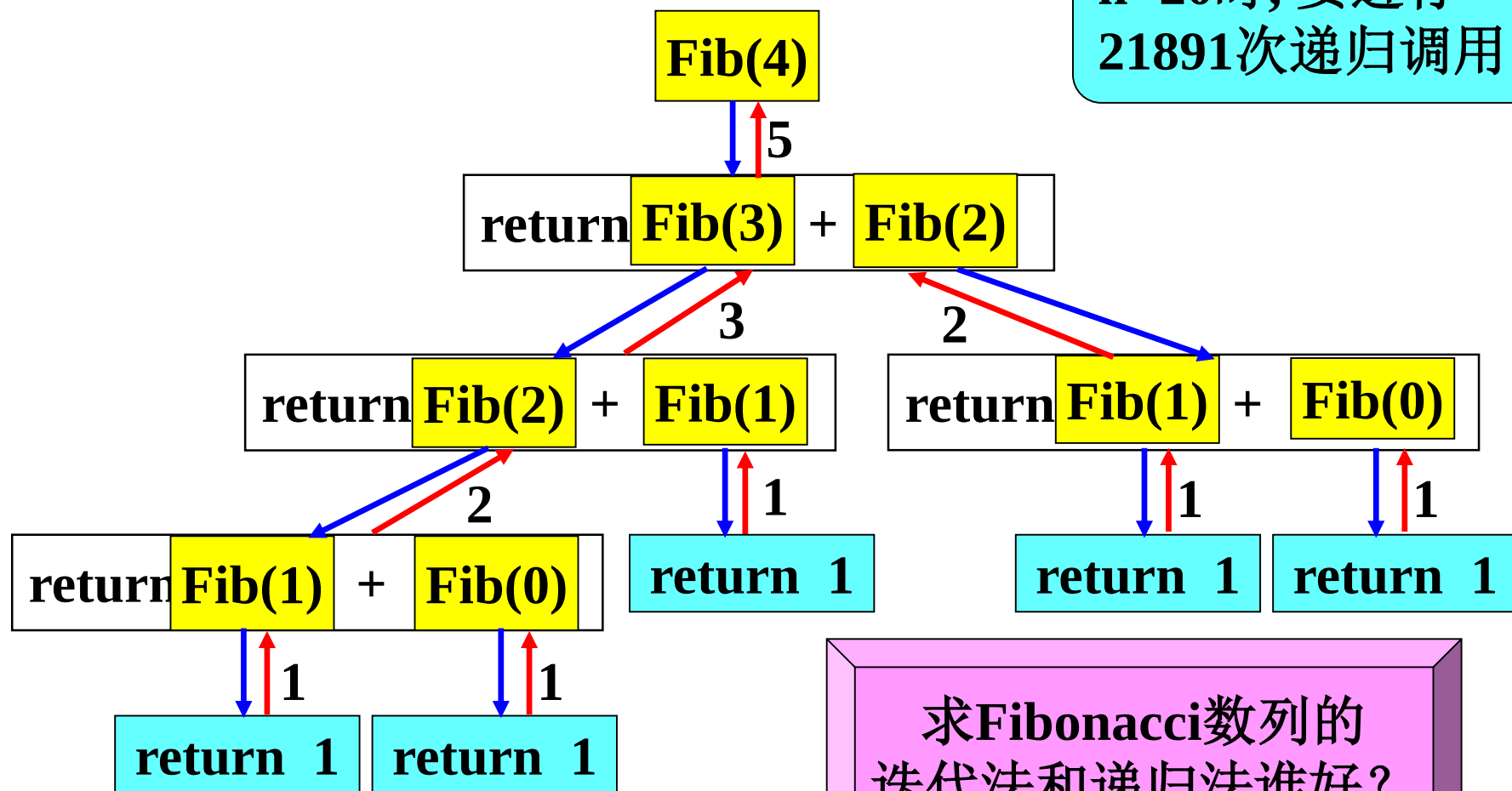
```
int Fib(int n)
{ if (n<0)
    { printf("error!"); exit(-1); }
  else
    { if (n <= 1) return 1;
      else return Fib(n-1)+Fib(n-2);
    }
}
```



5.5.1 递归函数

【例5.14】用递归函数求Fibonacci数列的第n项

n=20时, 要进行
21891次递归调用



求Fibonacci数列的
迭代法和递归法谁好?

5.5.1 递归函数

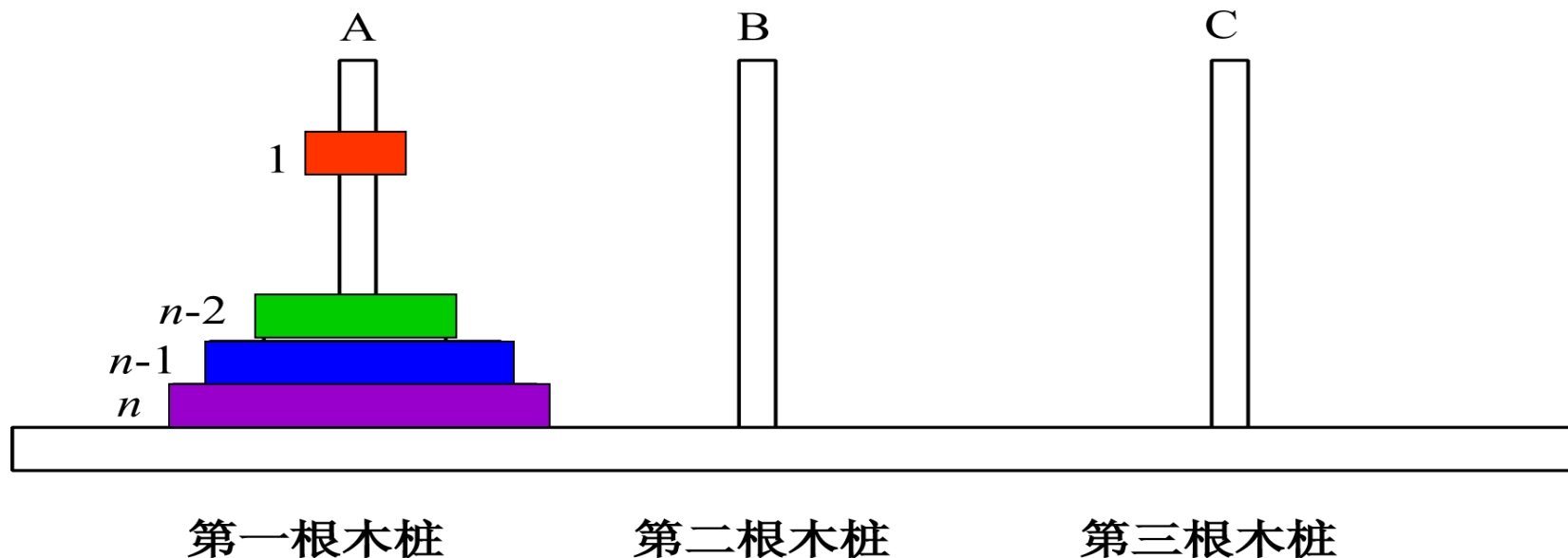
【例5.15】 Hanoi塔问题

问题描述： 设A, B, C是3个塔座。在塔座A上有 n 个圆盘, 这些圆盘自下而上, 由大到小的叠放。要求将A上的圆盘移到C上, 并仍按同样顺序叠放。移动圆盘应遵守以下规则:

规则1: 每次只能移动1个圆盘

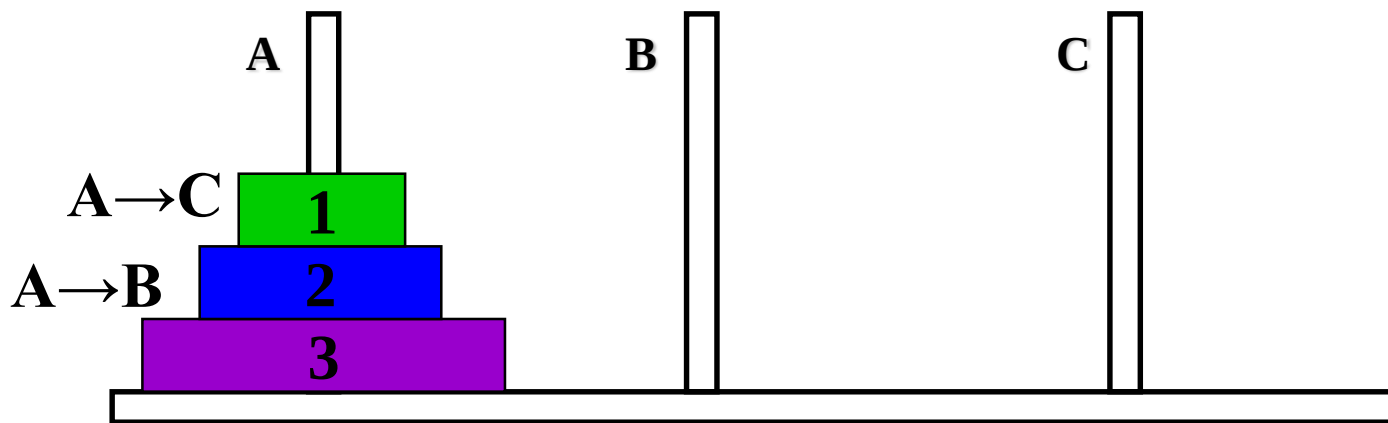
规则2: 任何时刻都不允许将大圆盘压在小圆盘之上

规则3: 在满足规则1和2的前提下, 可将圆盘移至任一塔座上



5.5.1 递归函数

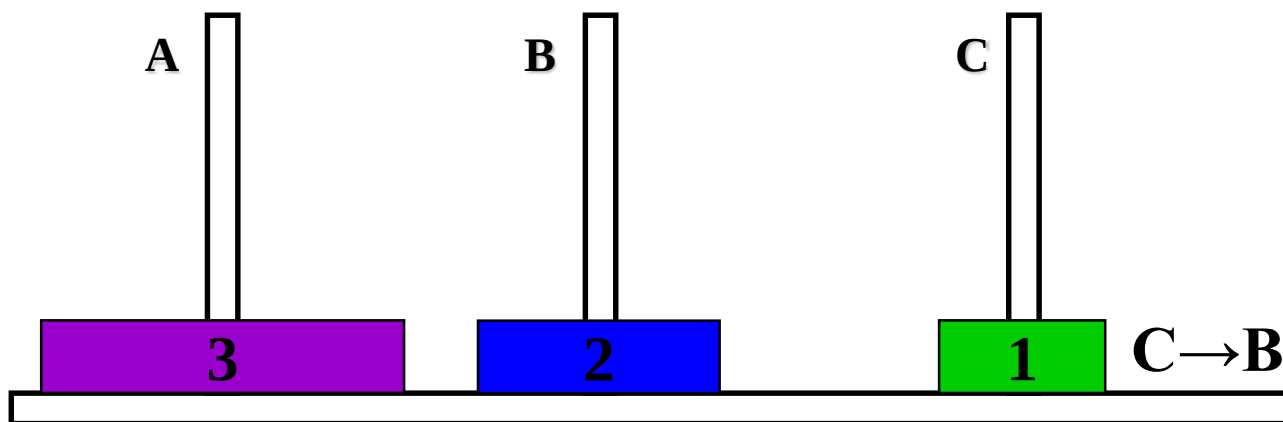
- Hanoi塔问题 3个圆盘的移动过程演示



1号盘: $A \rightarrow C$, 2号盘: $A \rightarrow B$

5.5.1 递归函数

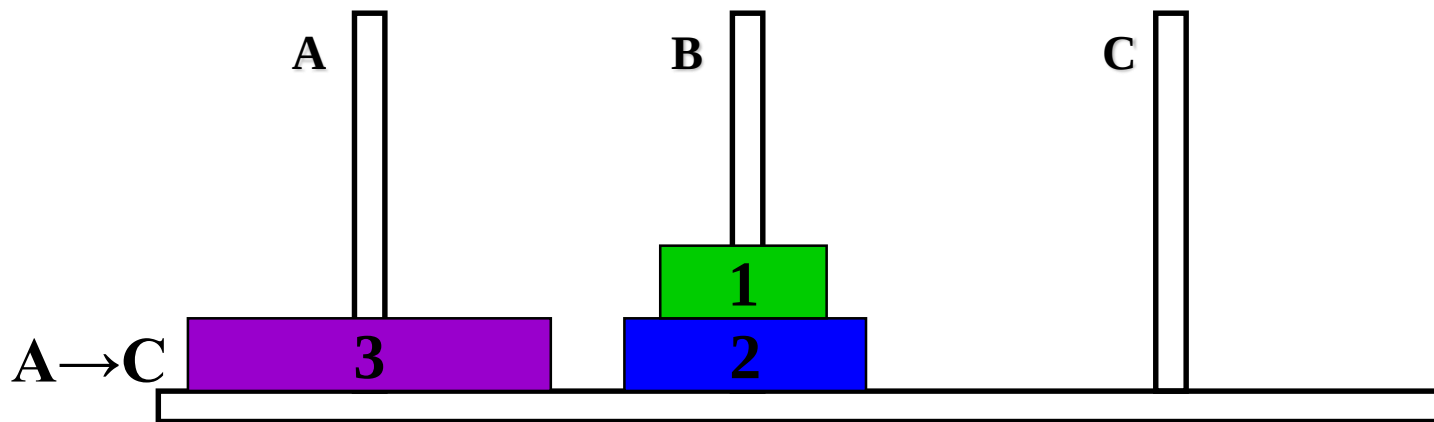
- Hanoi塔问题 3个圆盘的移动过程演示



1号盘: C → B

5.5.1 递归函数

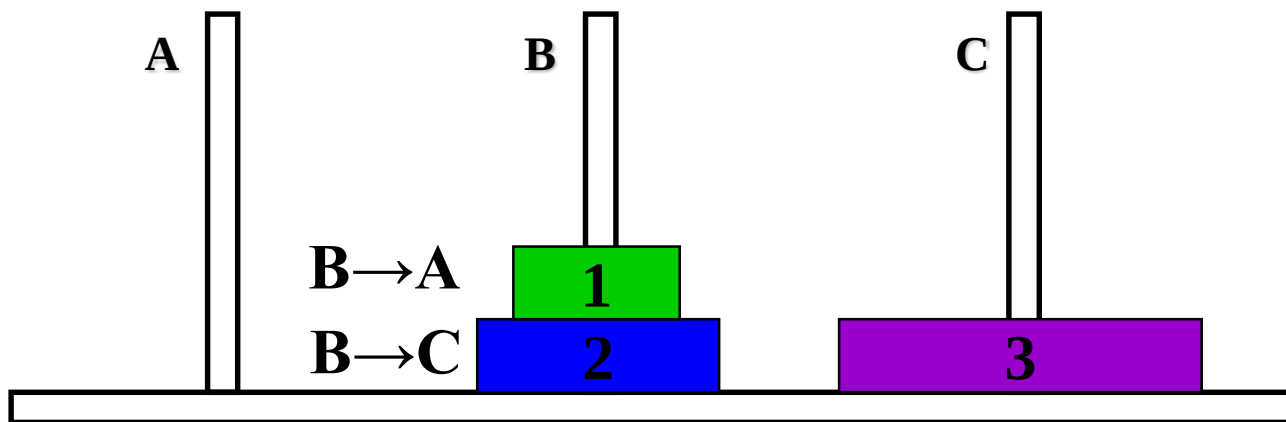
- Hanoi塔问题 3个圆盘的移动过程演示



3号盘: A → C

5.5.1 递归函数

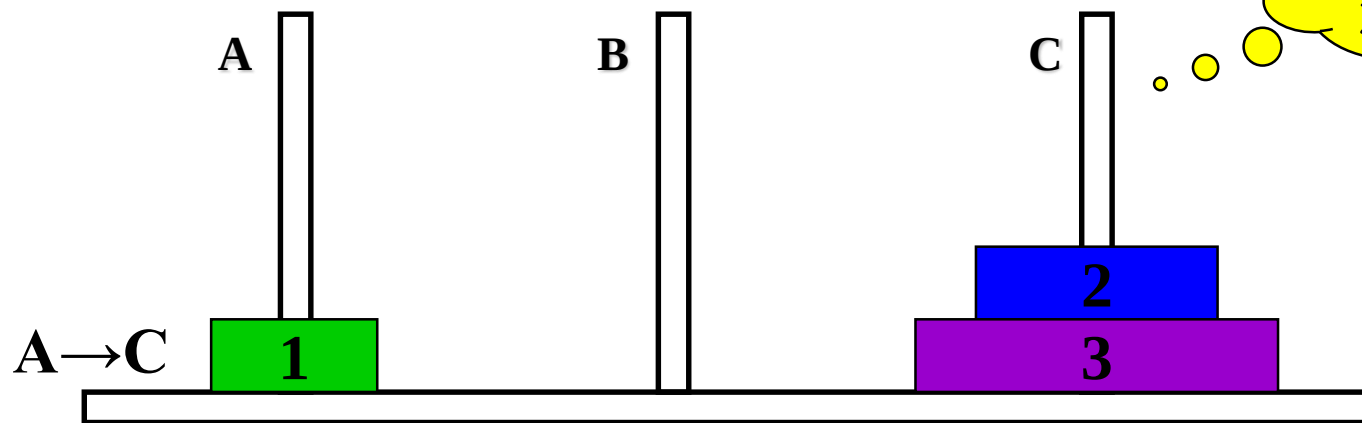
- Hanoi塔问题 3个圆盘的移动过程演示



1号盘: $B \rightarrow A$, 2号盘: $B \rightarrow C$

5.5.1 递归函数

● Hanoi塔问题 3个圆盘的移动过程演示



• 3个圆盘一共需要7次移动：

1号盘：A→C， 2号盘：A→B，

1号盘：C→B， 3号盘：A→C，

1号盘：B→A， 2号盘：B→C，

1号盘：A→C

n个圆盘需要 2^n-1 次移动

当 $n=64$ 时，需要1844亿亿次移动，若每次移动用1微秒，则需要60万年！

5.5.1 递归函数

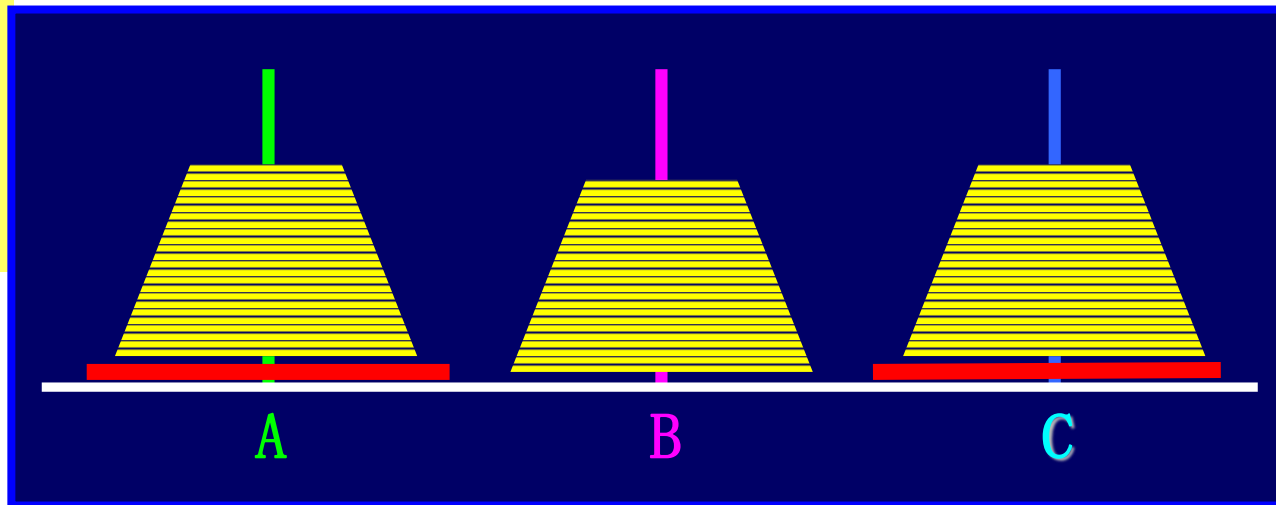
- n 个圆盘的移动方法:
 n 个圆盘分为2部分 $\left\{ \begin{array}{l} \text{上面}(n-1)\text{个圆盘} \\ \text{最下面的第}n\text{号圆盘} \end{array} \right.$

移动步骤:

- ① $(n-1)$ 个圆盘 $A \rightarrow B$
- ② 第 n 个圆盘 $A \rightarrow C$
- ③ $(n-1)$ 个圆盘 $B \rightarrow C$

$(n-1)$ 个圆盘
怎么移动?

递归何时结束?



第 n 个圆盘 $A \rightarrow C$ $n=1$

$\left\{ \begin{array}{l} (n-1)\text{个圆盘 } A \rightarrow B \\ \text{第}n\text{个圆盘 } A \rightarrow C \\ (n-1)\text{个圆盘 } B \rightarrow C \end{array} \right. \quad n>1$

5.5.1 递归函数

【例5.15】 Hanoi递归函数定义

// 将n个盘子从A移到C,借助于B

```
void Hanoi(int n, char a, char b, char c)
```

```
{ if (n==1)
```

```
    Move(n, a, c);
```

// 将第n个盘子从a移到c

```
else
```

```
{ Hanoi(n-1, a, c, b);
```

// 将n-1个盘子从a移到b,借助于c

```
    Move(n, a, c);
```

// 将第n个盘子从a移到c

```
    Hanoi(n-1, b, a, c);
```

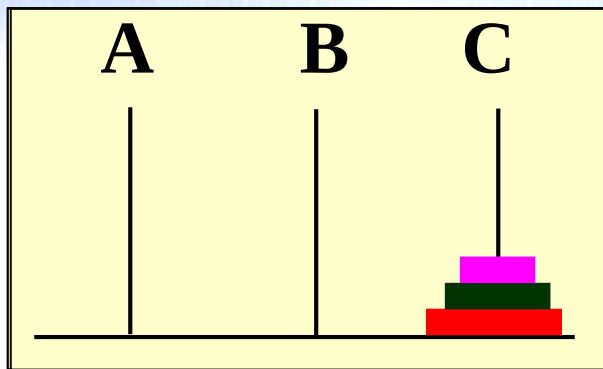
// 将n-1个盘子从b移到c,借助于a

```
}
```

```
}
```

请特别注意这3个参数的顺序

【例5.15】 递归过程演示



Hanoi(3, 'A', 'B', 'C')

```
{ if (n==1)
```

```
    Move(n, a, c);
```

```
else { Hanoi(2, a, c, b);
```

```
      Move(3, a, c);
```

```
      Hanoi(2, b, a, c);
```

```
}
```

```
}
```

Hanoi(2, 'A', 'C', 'B')

```
{ if (n==1)
```

```
    Move(n, a, c);
```

```
else
```

```
{ Hanoi(1, a, c, b);
```

```
  Move(2, a, c);
```

```
  Hanoi(1, b, a, c);
```

```
}
```

```
}
```

Hanoi(1, 'A', 'B', 'C')

```
{ if (n==1)
```

```
    Move(1, a, c);
```

```
}
```

Hanoi(1, 'C', 'A', 'B')

```
{ if (n==1)
```

```
    Move(1, a, c);
```

```
}
```

Hanoi(2, 'B', 'A', 'C')

```
{ if (n==1)
```

```
    Move(n, a, c);
```

```
else
```

```
{ Hanoi(1, a, c, b);
```

```
  Move(2, a, c);
```

```
  Hanoi(1, b, a, c);
```

```
}
```

```
}
```

Hanoi(1, 'B', 'C', 'A')

```
{ if (n==1)
```

```
    Move(1, a, c);
```

```
}
```

Hanoi(1, 'A', 'B', 'C')

```
{ if (n==1)
```

```
    Move(1, a, c);
```

```
}
```

5.5.2 分治算法

➤ 分治算法的基本思想就是“分而治之”。

将一个难以直接求解的复杂问题分解成若干个规模较小、相互独立，但类型相同的子问题，然后求解这些子问题，若子问题还比较复杂不能直接求解，则继续细分，直到子问题足够小能直接求解为止，最后将各子问题的解组合成原问题的解。

➤ 一个问题能够用分治法求解的要素是：

- ① 问题可以分解为若干个规模较小、相互独立，且与原问题类型相同的子问题；
- ② 子问题足够小时可以直接求解；
- ③ 可以将子问题的解组合成原问题的解。

5.5.2 分治算法

【例5.16】二分搜索问题

问题描述:

给定已排序的 n 个元素 $a[n]$, 在这 n 个元素中找出一特定元素 x

二分搜索的基本思想:

将 n 个元素分成大致相同的两半, 取中间元素 $a[n/2]$ 与 x 比较, 可能出现以下三种比较结果:

- ① 若 $x=a[n/2]$, 则找到 x , 停止搜索;
- ② 若 $x<a[n/2]$, 则在 $a[0] — a[n/2-1]$ 中继续搜索 x ;
- ③ 若 $x>a[n/2]$, 则在 $a[n/2+1] — a[n-1]$ 中继续搜索 x 。

5.5.2 分治算法

➤ 二分搜索实例：设在数组a中顺序放了以下9个元素

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]
-15	-6	0	7	9	23	54	82	101
③	②	③	④	①		②	③	④

检索 $x=9$, $9 = a[4]$, 一次比较就成功, 最好情况

检索 $x=-15$, $-15 < a[4]$, $-15 < a[1]$, $-15 = a[0]$, 3次比较, 成功

检索 $x=101$, $101 > a[4]$, $101 > a[6]$, $101 > a[7]$, $101 = a[8]$,
4次比较, 成功

检索 $x=8$, $8 < a[4]$, $8 > a[1]$, $8 > a[2]$, $8 > a[3]$, 4次比较, 不成功

5.5.2 分治算法

【例5.16】 二分搜索用迭代法求解

```
int Search( int a[ ], int x, int left, int right )
```

```
{  
    int mid;  
    while( left<=right )  
    { mid=(left+right)/2;  
      if(x==a[mid])  
          return(mid);  
      else  
          if(x<a[mid])  
              right=mid-1;  
          else left=mid+1;  
    }  
    return(-1);  
}
```

```
int main( )  
{ int i, x, a[10], result;  
  for(i=0; i<10; i++)  
      scanf("%d", &a[i]);  
  printf("input x:");  
  scanf("%d", &x);  
  result=Search(a, x, 0, 9);  
  if ( result== -1)  
      printf("%d is not found.\n", x);  
  else  
      printf("Find %d==a[%d]\n", x, result);  
  return 0;  
}
```



5.5.2 分治算法

【例5.16】二分搜索用分治法求解，递归函数定义如下：

```
int Search(int a[ ], int x, int left, int right)
{
    int mid;
    if (left > right) return(-1); //left > right时表示搜索失败，返回-1
    else
    {
        mid = (left + right) / 2; //计算中间元素的下标
        if (x == a[mid]) return(mid); //找到x，返回下标值
        else //x不等于中间元素的情况
        {
            if (x < a[mid]) //x小于中间元素的情况
                Search(a, x, left, mid-1); //递归调用
            else //x大于中间元素的情况
                Search(a, x, mid+1, right); //递归调用
        }
    }
}
```

5.6 局部变量和全局变量

- C语言中的变量有作用域和生存周期的概念
 - 变量的作用域指出了变量在什么范围内有效，
 - 变量的生存周期决定了变量的存活期，从系统为变量分配内存空间开始，到系统收回内存空间为止。
- 变量的作用域描述了变量的空间有效性，生存周期描述了变量的时间有效性。

变量的生存周期涵盖了变量的作用域，因为变量有效必然意味着变量是存活着的，而反过来则未必，即变量存活不一定就代表着变量必然可以使用。
- C变量按作用域范围可分为两种：**局部变量**和**全局变量**。



5.6.1 局部变量

- 局部变量是指在一个函数（或复合语句）内部定义的变量，也称为内部变量。
- 局部变量有两层含义：
 - ① 一个函数（或复合语句）中定义的变量只在本函数(或复合语句)范围内有效，不能在定义函数(或复合语句)外使用。
 - ② 局部变量在每次函数调用时分配存储空间，在函数返回时释放存储空间。

5.6.1 局部变量

```
#include <stdio.h>
```

```
#define N 6
```

```
void sort(int a[N]);
```

```
int main( )
```

```
{ int b[N], i;
```

main函数中的局部变量:b和 i

```
    for( i=0; i<N; i++) scanf("%d", &b[i] );
```

```
    sort( b );
```

```
    for( i=0; i<N; i++) printf( "%3d", b[i] );
```

```
    return 0;
```

```
}
```

```
void sort (int a[N])
```

参数a也是局部变量

```
{ int i, j;
```

sort函数中的局部变量: i, j

```
    for( i=0; i<N-1; i++)
```

```
        for( j=0; j<N-1-i; j++)
```

```
            if ( a[j]>a[j+1] )
```

```
                { int t;
```

t 是复合语句中的局部变量,
只能以下在3个赋值语句中使用

```
                    t=a[j];
```

```
                    a[j]=a[j+1];
```

```
                    a[j+1]=t;
```

```
                }
```

```
}
```

5.6.1 局部变量

➤ 局部变量使用说明:

- ① 局部变量只在本函数的范围内有效，包括在main函数中定义的变量也只能在main函数中使用。
- ② 不同函数中可以使用相同名字的变量，因为其作用域都是自身函数。它们代表不同的对象，占用不同的内存单元，互相独立。
- ③ 函数的形式参数也是局部变量。
- ④ 在复合语句中可以定义变量，其作用域只在本复合语句内。

5.6.1 局部变量

【例5.18】在复合语句中定义局部变量。

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int i, a=0;           //定义第1个局部变量a
```

```
    for (i=1; i<=2; i++)
```

```
    { int a;              //定义第2个局部变量a
```

```
        a=i+1;
```

```
        printf("i=%d, a=%d\n", i, a);
```

```
    }
```

```
    printf("i=%d, a=%d\n", i, a);
```

```
    return 0;
```

```
}
```



程序中两个同名局部变量a，第2个变量a是在复合语句中定义，其作用域是本复合语句，而第1个变量a在此复合语句中不起作用。

```
i=1, a=2
i=2, a=3
i=3, a=0
```

5.6.2 全局变量

- 全局变量是指在函数外定义的变量，因此也称为外部变量。
- 全局变量在程序开始运行时分配存储空间，在程序结束时释放存储空间。
- 全局变量的作用域：
从源程序文件中定义该变量的位置开始，到本源程序文件结束，即位于全局变量定义后的所有函数都可以使用该变量。

5.6.2 全局变量

```
#include <stdio.h>
```

```
int p=1, q=5;
```

```
double f1( int a )
```

```
{ double r ;
```

```
    :
```

```
}
```

```
int s[10];
```

```
int f2( int b , int c );
```

```
{ int sum ;
```

```
    :
```

```
}
```

```
float m, n ;
```

```
int main( )
```

```
{ float x , y ;
```

```
    :
```

```
}
```

全局变量m和
n的有效范围

全局变量数组
s的有效范围

全局变量p和
q的有效范围

5.6.2 全局变量

➤ 全局变量使用说明:

① 由于全局变量定义后，在任何函数中都可以访问。

可以利用全局变量从函数中得到一个以上的返回值，但同时也容易造成数据值的混乱，使用时必须清楚地知道全局变量值的变化情况。

如果使用全局变量过多，将难以清楚地判断程序执行过程中全局变量的值，会降低程序的清晰性。

5.6.2 全局变量

【例5.19】编写函数求某班级学生成绩的最高分、最低分和平均分
分析：由于题目要求得到最高分、最低分和平均分三个值，但一个函数最多有一个返回值，为解决这个问题，可以使用全局变量来实现。

```
#include<stdio.h>

int g_max, g_min; //定义全局变量g_max, g_min
double g_ave;     //定义全局变量g_ave
void count(int a[], int n);
int main()
{ int i, n, s[50];
  printf("请输入班级人数:");
  scanf("%d", &n);
  for(i=0; i<n; i++) //输入每个学生的成绩
    scanf("%d", &s[i]);
  count(s, n); //调用count函数
  printf("平均分为%.2lf,最高分为%d,最低分为%d\n",
        g_ave,g_max, g_min);
  return 0;
}
```

5.6.2 全局变量

【例5.19】编写函数求某班级学生成绩的最高分、最低分和平均分

```
void count(int a[], int n)    //定义count函数
{
    double fSum=a[0];        // fSum初始化为a[0]
    g_max=g_min=a[0];        //全局变量g_max, g_min赋初值为a[0]
    int i;
    for(i=1; i<n; i++)
    {   if(a[i]>g_max)
        g_max=a[i];          //若a[i]大于g_max, 则更新g_max
        else
            if(a[i]<g_min) g_min=a[i]; //若a[i]小于g_min, 则更新g_min
        fSum=fSum+a[i];      //计算总分
    }
    g_ave=fSum/n;            //计算平均分
}
```



5.6.2 全局变量

【例5.20】 程序调用对全局变量值的影响。

```
#include<stdio.h>
int m=1;
void fun1(int x)
{
    m=m+x;
    printf("m=%d\n", m++);
}
void fun2(int y)
{
    m=m+y;
    printf("m=%d\n", ++m);
}
```

```
int main()
{
    int a=2;
    m=m+a;
    printf("m=%d\t", m);
    fun1(a);
    printf("m=%d\t", m);
    fun2(a);
    return 0;
}
```

程序输出：

```
m=3    m=5
m=6    m=9
```

5.6.2 全局变量

➤ 全局变量使用说明:

- ② 若程序中全局变量与局部变量同名，且同时有效，则以局部变量优先。即在局部变量的作用范围内，全局变量不起作用。

【例5.21】局部变量与全局变量同名的情况。

```
#include <stdio.h>
```

```
int x=10;           //定义全局变量x
```

```
void fun(void)
```

```
{ int x=1;          //定义局部变量x
```

```
    x=x+1;
```

```
    printf("x=%d\n", x );
```

```
}
```

```
int main( )
```

```
{ x=x+1;
```

```
    printf("x=%d\n", x);
```

```
    fun( );
```

```
    return 0;
```

```
}
```

输出结果:

x=11

x=2

5.7 变量的存储类别

5.7.1 auto变量

C语言中的变量和函数有两个属性：数据类型和数据的存储类别。

数据类型：即整型、浮点型、字符型等。

存储类别：指数据在内存中的存储方式。

变量的存储类别分4种

- auto—自动的
- static—静态的
- register—寄存器的
- extern—外部的

5.7.1 auto变量

函数中的局部变量, 如不特别声明变量的存储类别, 都是自动变量

自动变量的特点是: 在调用函数时系统会自动给自动变量分配存储空间, 在函数调用结束时则自动释放这些存储空间。

自动变量的声明格式: **auto 数据类型 变量名;**

例如:

```
int main( )
```

```
{
```

```
    int a=3, b=5;
```

a和b也是自动变量, auto可省略不写

```
    auto int c;    //明确定义自动变量c
```

```
    c=a+b;
```

```
    printf("c=%d\n", c);
```

```
    return 0;
```

```
}
```


5.7.2 static变量

如果希望函数中局部变量的值在函数调用结束后不消失而保留原值(即局部变量所占用的存储空间不被释放),这时就应用关键字static声明该局部变量为“静态局部变量”。

静态局部变量的声明格式: **static 数据类型 变量名;**

【例5.22】考察静态局部变量的值。

```
#include<stdio.h>
```

```
void fun(void)
```

```
{ int a=1;
```

```
    static int b=1; //定义静态局部变量b, 并初始化
```

```
    a=a+1; b=b+1;
```

```
    printf("a=%d, b=%d\n", a, b);
```

```
}
```

```
int main( )
```

```
{ int i;
```

```
    for(i=1;i<=3;i++)
```

```
        { printf("第%d次调用函数: ", i); fun( ); }
```

```
    return 0;
```

```
}
```

程序运行结果:

第1次调用函数: a=2, b=2

第2次调用函数: a=2, b=3

第3次调用函数: a=2, b=4

5.7.2 static变量

➤ 静态局部变量与自动变量的区别：

- ① 静态局部变量属于静态存储方式，在静态存储区内分配存储空间，在程序整个运行期间都不释放。而自动变量属于动态存储方式，其存储空间在函数调用结束后被释放。
- ② 静态局部变量在编译时只进行一次初始化操作，以后的每次函数调用都不再进行初始化；而自动变量的初始化在每一次的函数调用中都会重复执行。
- ③ 静态局部变量如果不进行初始化，编译时系统会自动赋初值为0或空字符。自动变量如果不初始化，则它的值是随机值。
- ④ 虽然静态局部变量所占的存储空间在函数调用结束后仍然存在，但是其他的函数不能引用它。而自动变量所占的存储空间在函数调用结束后将被释放。

5.7.2 static变量

【例5.23】编写一个售票程序，用一个函数来销售火车票，初始时有100张票，函数参数表示销售火车票的数量，每执行一次销售函数，就减掉参数对应的数量。

```
#include<stdio.h>
#include<stdlib.h>
int saleTicket(int num)
{
    static int total=100;    //一共有100张火车票
    if(total-num>=0)
    {
        total=total-num;    //减掉相应的票数
        return 1;
    }
    else return 0;          //票不够了
}
```

5.7.2 static变量

【例5.23】售票程序

```
int main()
{ int num; char ch;
  do
  { printf("请输入您要的票数: \n");
    scanf("%d", &num);  getchar();
    if( saleTicket(num)!=0 ) printf("买了%d张票! \n", num);
    else
    { printf("对不起, 票已售空或余票数不足! \n");
      exit(0);
    }
    printf("继续买票, 请录入Y,退出请录入N!\n");
    ch=getchar();
  }while(ch=='Y' || ch=='y');
  return 0;
}
```



5.7.2 static变量

【例5.24】输出1到4的阶乘值。

```
#include<stdio.h>
int Factorial (int n);
int main( )
{
    for(int i=1; i<=4; i++)
        printf(" %d!=%d\n", i, Factorial(i) );
    return 0;
}
```

```
int Factorial (int n)
{
    int f=1;
    for(i=1; i<=n; i++)
        f=f*i;
    return(f);
}
```

改写

```
int Factorial (int n)
{
    static int f=1;
    f=f*n;
    return(f);
}
```

使用静态局部变量存在的问题：

- ① 占用较多的内存空间，因为static变量在程序运行期间一直要占用内存；
- ② 会降低程序的可读性，因为函数调用次数太多时，很难确定static变量的当前值。

5.7.3 register变量

- 为提高程序的运行速度，可将使用十分频繁的局部变量说明为**寄存器变量**，将其存储在 CPU 的寄存器中。

寄存器变量的声明格式：**register 数据类型 变量名;**

例如：求n! 。

```
#include<stdio.h>
int main( )
{ int n;
  register int i, f=1;
  scanf("%d", &n);
  for(i=1; i<=n; i++)
    f=f*i;
  printf("%d!=%d\n", n, f);
  return 0;
}
```

说明：

- ① 只有自动变量和形式参数可以作为寄存器变量；
- ② 因一个计算机系统中的寄存器数目是有限的，不能定义任意多个寄存器变量；
- ③ 静态局部变量不能定义为寄存器变量。

5.7.4 extern变量

- 外部变量（即全局变量）是在函数的外部定义的，它的作用域为从变量定义处开始到本程序文件的末尾。
- 如果在外部变量定义点之前的函数想引用该全局变量，则应该在函数内用关键字**extern**对该变量作“外部变量声明”，表示该变量是一个已经定义的外部变量。

◆ 注意

extern与前面介绍的三种存储类别不同，前三种都是在变量定义时在数据类型前加上关键字**auto**、**static**或**register**，而**extern**是对已经定义好的全局变量进行声明，并不是在变量定义时加关键字**extern**。

5.7.4 extern变量

【例5.25】用extern声明外部变量，扩展程序文件中全局变量的作用域。

```
#include<stdio.h>
int Max(int x, int y);
int main()
```

```
{ extern int g_a, g_b;
```

```
    printf("max=%d\n", Max(g_a, g_b));
```

```
    return 0;
```

```
}
```

```
int g_a=15, g_b=8;
```

```
int Max(int x, int y)
```

```
{ int z;
```

```
    z=x>y?x:y;
```

```
    return(z);
```

```
}
```

用extern声明外部变量时，变量的数据类型可以写也可以省略不写。

例如：extern int g_a, g_b; 也可以写成
extern g_a, g_b;

//声明g_a、g_b是外部变量

//定义全局变量g_a、g_b



**培养创新思维
提升编程能力**